Week 7

# Grids, Recursion and Fractal Art

Level 3: Advanced

Functions that call themselves; worlds made of grids.

## What you will learn

- What **recursion** is: a function that calls itself on a smaller problem.
- Why every recursive function needs a **base case**.
- How to store a game map or a picture in a **grid** (a list of lists).
- How to draw a **fractal tree** and build a **paint bucket** tool.

## The big idea: functions that call themselves

Open a Russian doll and you find a smaller doll. Open that one: an even smaller doll. You keep doing *the same job* until you reach the tiny solid doll – then you stop.

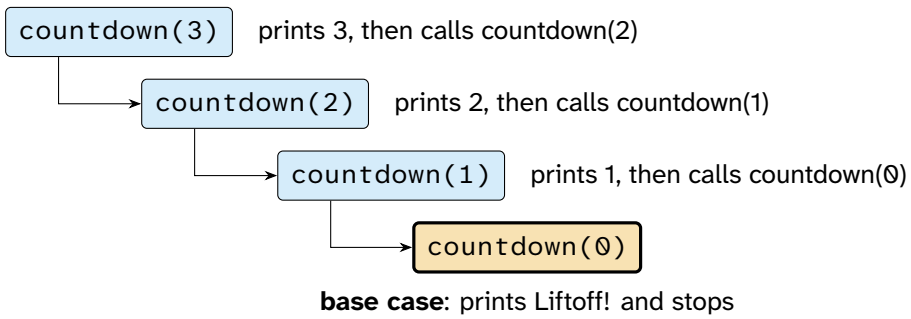
A **recursive** function works the same way. It has two parts:

- the **base case**: the tiny problem you can answer straight away – *stop here*;
- the **recursive case**: do one small step, then call yourself on a *smaller* problem.

**countdown: a recursive function**

```
def countdown(n):
    if n <= 0:              # base case: stop
        print("Liftoff!")
        return
    print(n)                # one small step
    countdown(n - 1)        # the same job on a smaller number
```

When you call `countdown(3)`, Python stacks up the calls like plates. Each call waits for the smaller one to finish:




**Bug hunt: No base case = never-ending dolls**

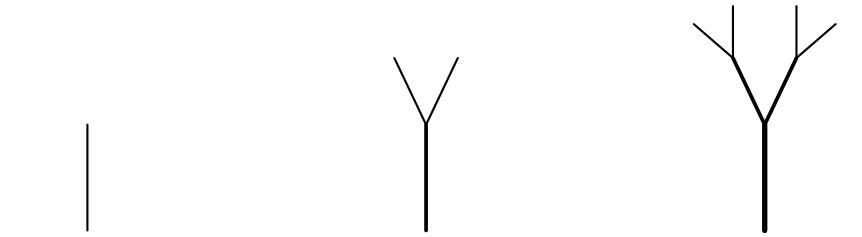
If you forget the base case, the function calls itself forever. Python stops it after about `1000` calls with `RecursionError: maximum recursion depth exceeded`. When you see that error, look for the base case first. Also check that each call is on a *smaller* problem (`n - 1`, not `n`).

**Predict!** What does `stars(3)` print? Write it before you run it.

```
def stars(n):
    if n == 0:
        return
    print("*" * n)
    stars(n - 1)
```

## Fractals: pictures made of smaller copies

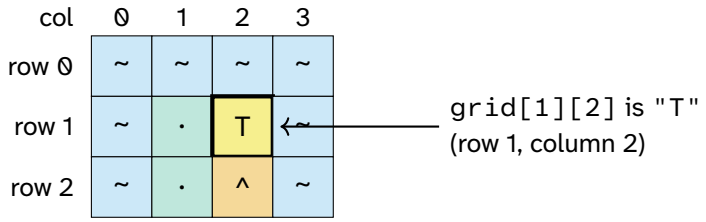
A **fractal** is a shape made of smaller copies of itself. A tree is a trunk with two smaller trees on top. Each smaller tree is a trunk with two even smaller trees on top… until the **depth** reaches `0`.



depth 1: 1 line | depth 2: 3 lines | depth 3: 7 lines

## Grids: worlds made of rows

A **grid** is a list of rows, and each row is a list of cells. You find a cell with `grid[row][col]`: first go *down* to the row, then *across* to the column. Both start at `0`.




You can make a grid from strings: `grid = [list(row) for row in ["~~~~", "~.T~", "~.^~"]]`. Change a cell with `grid[2][1] = "T"`. The **neighbors** of a cell are the cells up, down, left and right of it (watch out for the edges!).

A **paint bucket** (flood fill) uses recursion on a grid: paint this cell, then ask each neighbor to do the same – but stop at the edge, at walls, and at cells you have already painted. Those are the base cases.

### Exercise 7.1: Recursion warm-up

Folder: `week07/ex-countdown-sum`. Read `countdown()`. Then finish `sum_to(n)` (so `sum_to(5)` is 15) and `reverse(word)` (so `reverse("turtle")` is `"eltrut"`) using recursion. Check with `check_my_work.py`.

**Write the base case of `sum_to`:**

### Exercise 7.2: Fractal tree

Folder: `week07/ex-fractal-tree`. Inside `branch()`, after the trunk, add: turn left, draw a smaller tree, turn right twice as far, draw another smaller tree, turn back. Run it and fill in the table.

| Depth | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Number of lines | 1 | 3 | ____ | ____ | ____ |

What pattern do you see?

### Exercise 7.3: Paint bucket

Folder: `week07/ex-paint-bucket`. Finish `count_tiles(grid, symbol)` with two loops, then the flood fill: add the base case “stop if this cell is not the old color” and four recursive calls (up,

down, left, right). Run it, press keys **1–5** to choose a paint and click inside a shape. Every paint has a letter as well as a color.
**Which cells does the paint stop at?**

**Tip: Watch the calls pile up**

In Thonny, click the bug icon (Debug) and press F7 (Step into) again and again on `sum_to(3)`. Each new call opens a new box – that is the call stack.

**Vocabulary box**

**recursion** When a function calls itself to solve a smaller copy of the same problem.

**base case** The simplest case, answered without calling the function again. It stops the recursion.

**recursive case** The part that does a small step and calls the function on a smaller problem.

**RecursionError** Python's error when a function calls itself too many times – usually a missing base case.

**fractal** A shape made of smaller copies of itself, like a tree, a fern or a snowflake.

**grid (2D list)** A list of rows, where each row is a list. `grid[row][col]` is one cell.

**flood fill** Filling every touching cell of the same color, like a paint bucket tool.

**Reflection**

Where in nature have you seen something made of smaller copies of itself?

Explain the base case to someone who has never coded:

**Week 7** Level 3: Advanced

# Grids, Recursion and Fractal Art

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

## ★ Core (everyone): Treasure map (about 10 minutes)

Open `3-advanced/exercises/week07/hw-treasure-map/starter.py` in Thonny and press F5. The map is a grid: ~ water, ^ mountain, . grass, T treasure, S start.

1. Finish `count_tiles(grid, tile)`. Hint: a list has `.count()`, e.g. `["a", "b", "a"].count("a")` is 2.
2. Finish `neighbors(grid, row, col)`: the cells up, down, left and right – but never off the edge of the map.
3. Run `check_my_work.py` until every check passes.

How many neighbors does a corner cell have? _____ A cell in the middle? _____

## ★★ Challenge (want more?): Koch snowflake

Open `week07/hw-koch-snowflake/starter.py`. A Koch line of depth 0 is a straight line; a Koch line of depth $n$ is **four** Koch lines of depth $n-1$, each a third as long, with turns of 60, 120, 60 degrees making a bump: `___/\___`. Replace the straight line in the recursive case and run it.
Fill in: depth 1 = 4 lines per side, depth 2 = _____, depth 3 = _____. What is the base case?

## Creative (make it yours): My own fractal

Change the fractal tree from class (ANGLE, SHRINK, colors, three branches, a leaf dot in the base case) or draw a recursive pattern of your own: squares inside squares, a spiral, circles on circles. Add a comment at the top: `# Based on: ...` to credit the idea. Sketch it here:

**Self-check: tick each box when it is true**

- ☐ I can point to the base case and the recursive case in a recursive function.
- ☐ I know `grid[row][col]` means row first, then column, both from `0`.
- ☐ My `neighbors` function never returns a cell off the map.
- ☐ If I see `RecursionError`, I check the base case first.